capital

Custom Table Development

Software Version 2015.1

February 2016

© 2016 **Mentor Graphics Corporation**
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS CORPORATION HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

RESTRICTED RIGHTS LEGEND 03/97

U.S. Government Restricted Rights. The SOFTWARE and documentation have been developed entirely at private expense and are commercial computer software provided with restricted rights. Use, duplication or disclosure by the U.S. Government or a U.S. Government subcontractor is subject to the restrictions set forth in the license agreement provided with the software pursuant to DFARS 227.7202- 3(a) or as set forth in subparagraph (c)(1) and (2) of the Commercial Computer Software - Restricted Rights clause at FAR 52.227-19, as applicable.

Contractor/manufacturer is:
Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777.
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: www.mentor.com
SupportNet: www.mentor.com/supportnet
Contact Your Technical Writer: www.mentor.com/supportnet/documentation/reply_form.cfm

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other third parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the respective third-party owner. The use herein of a third- party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at:
www.mentor.com/terms_conditions/trademarks.cfm.

End-User License Agreement: You can print a copy of the End-User License Agreement from:
www.mentor.com/terms_conditions/enduser.cfm.

Table Of Contents

Introduction **2**
Interfaces **3**
IXTable 3
IXHarnessTable 3
IXLogicTable 3
IXIntegratorTable 4
IXTableData 4
Guidance **5**
Example **6**

Introduction

Table plugins allows custom development of tables using the extensibility architecture of Capital.

A table in Capital consists of two elements: the data and the header. Any table plugin is responsible for providing the data which will be displayed in the regular cells of the table. Additionally the plugin is responsible for providing the header values that will be displayed. A maximum of three headers can be displayed in a table. Headers can be independently styled from data cells using the Capital styling editor.

Interfaces

IXTable

This is the main interface of a table plugin and is not to be directly implemented. The developer of a t able plugin should instead implement any of the sub interfaces of this interface. The sub interfaces of IXTable are IXHarnessTable, IXIntegratorTable and IXLogicTable.

The methods in IXTable are:

1. **Class<? extends IXObject> getTableContext() –** This method returns the table context of the table plugin. Each table plugin is written for a specific entity, i.e. connector, splice or , design. This method should return the class of the interface for which the table plugin was written. For example, a connector is represented by the IXConnector interface. A table plugin written for a connector should return IXConnector as the table context. Similarly, a table plugin for a splice should return IXSplice and table plugin for harness design should return IXHarnessDesign. A table plugin receives the IXObject (Capital Extensibility object) of the type defined in this method.
2. **void initialize(IXObject xObject)** – This method is called once for every initialization and refresh of a table plugin. The IXObject (Capital Extensibility object) passed to this method depends on the table context. A table plugin written for connector will receive a IXConnector object. Similarly, a table plugin written for Splice will receive IXSplice and a table plugin written for harness design will receive IXHarnessDesign in this method call.
3. **IXTableData populate(IXObject xObject)** – This method is called to populate the table model. The table data model is represented by the IXTableData interface and is essentially a two dimensional structure containing header values and data values. The IXObject passed in this call is the same as that in the initialize call.

IXHarnessTable

The sub interface of IXTable. Plugin developer should implement this interface for table plugins written for Capital HarnessXC.

IXLogicTable

The sub interface of IXTable. The plugin developer should implement this interface for table plugins written for Capital Logic.

IXIntegratorTable

The sub interface of IXTable. The plugin developer should implement this interface for table plugins written for Capital Integrator. In 2009.1, table plugins for Capital Integrator are not supported.

IXTableData

This interface represents a table data model of a table plugin. Table plugin developers should implement this interface and return as part of IXTable.populate(IXObject xObject) method call. This interface models the data to be displayed in a table. A table in Capital HarnessXC is represented by two parts – the table data and table header. The table data represents the normal cells in a table. The table header represents the multiple column headers that a table can have.

Methods in IXTableData:

1. **int getColumnCount()** – This method returns the number of columns in a table plugin. Developers of table plugin should return the number of columns to be displayed in the rendered table. Number of columns in table header and table data are same.
2. **int getRowCount()** – This method returns the number of data rows in a table. As described earlier, data rows in a table are different from header rows in a table.
3. **int getHeaderCount() –** This method returns the number of header rows in a table. In Capital HarnessXC, a table column can have multiple headers.
4. **Object getCellValueAt(int rowIndex, int colIndex) –** This method returns table data at a particular cell. It should avoid returning a null value. An empty string or a placeholder character should be returned in case the value of a cell is null.
5. **Object getHeaderValueAt(int rowIndex, int colIndex) –** This method returns a table header value at a particular cell in the header rows. An empty string or a placeholder character should be returned in case the value of a cell is null.

Guidance

It is fairly easy to write a table plugin in Capital HarnessXC. Following are some useful tips for developers:

- Do not try to cache the IXObject in the table plugin. Capital Harness Extensibility architecture will supply a fresh IXObject for every call to IXTable.initialize(IXObject xObject) and IXTable.populate(IXObject xObject) method call. Though, IXObject passed to both the methods is guaranteed to be same.
- Some of the tables will require a complex hierarchy of IXObjects. Construct this hierarchy in the initialize method of IXTable.
- Try to make implementation of IXTableData as inner class of IXTable implementation. This will help in utilizing any calculation/object hierarchy already constructed in the IXTable implementation.

Example

You now have a fair idea about the building blocks of a plugin table, the interfaces to be used and the methods to be implemented. To make the picture much clearer and to help you get started right away, take a look at the sample plugin table that we have included for your understanding. A quick walk-through of this sample will help you understand and implement your own plugin tables which suit your requirement.

The example that we have provided tries to identify all the wires on a harness design and displays the attributes of each wire found. The special behavior of this plugin is that it identifies the wire that is connected to a centre-stripped wire (at the centre–strip splice) , and displays the name of the centre-stripped wire in the 'From'/'To' column as applicable. Since this plugin table works on a harness design within the Capital HarnessXC product, it has to implement *IXHarnessTable* and return *IXHarnessDesign* as the table context.

Here is the table generated by the example. The wire '*wire-CSSp1'* is a wire connected to a centre stripped wire namely '*wire-Main'* at the centre strip splice. Similarly , wire '*wire-CCSp2'* is connected to the centre stripped wire '*wire-2'*. The table also shows that wire '*wire-Spl'* is connected to a regular splice called '*SplSymbPart*.

Wire No	Col	Size	From	Cav	To	Cav	Option
wire-CSSp1	B	0.5	wire-Main	L	Conn_LT	Cavity-1	
wire-Main		1.0	Conn_LE	Termination1	Conn_RE	Cavity-1	
wire-Spl	R	0.5	Conn-RT	Cavity-2	SplSymbPart	L	
wire-simpmc-1		1.0	Conn_RE	Cavity-5	Conn_LE	Termination9	
wire-simpmc-2		1.0	Conn_RE	Cavity-5	Conn_LE	Termination8	
wire-CSSp2		1.0	wire-2	L	Conn-RT	Cavity-1	
wire-2		0.5	Conn-B	Termination1	Conn_RE	Cavity-1	

This example can be found in the following location within the Capital installation:

doc/plugin/examples/Java/src/com/example/plugin/Table/

- **CentreStripWireTableplugin.java** The main plugin table class.
  This class implements all the methods mandatory for writing a plugin table.

```
import com.mentor.chs.plugin.table.IXHarnessTable;
import com.mentor.chs.plugin.table.IXTableData;
import com.mentor.chs.api.*;

import java.util.Set;
```

```
public class CentreStripWireTableplugin implements IXHarnessTable
{
    private IXHarnessDesign design;
    private IXTableData m_xTableData;
    private String[][] data;

    public String getDescription()
    {
        return "Sample table plug-in for Wires & Centre Strip Info ";
    }


    public String getName()
    {
        return "Wires Table Plugin with centre strip  info";
    }


    public String getVersion() {
        return "1.0";
    }

    public Class getTableContext()
    {
        return IXHarnessDesign.class;
    }


    public void initialize(IXObject xObject)
    {
        try {
                if (xObject instanceof IXHarnessDesign)
                                {
                                        design = (IXHarnessDesign) xObject;
                                }
                m_xTableData = new CentreStripWireTableplugin.XTableData();
            } catch (Exception e)
            {
             e.printStackTrace();
            }
    }

    public IXTableData populate(IXObject xObject)
    {
        return m_xTableData;
    }

    class XTableData implements IXTableData {

        XTableData()
        {
            //get all the wires
             Set<IXWire> wires = design.getConnectivity().getWires();
             data = new String[getRowCount()][getColumnCount()];

             int i = 0;
             for (IXWire wire : wires)
             {

                 if (wire != null)
```

```
{
    data[i][0] = wire.getAttribute("Name");
    data[i][1] = wire.getAttribute("WireColor");
    data[i][2] = wire.getAttribute("WireCSA");

    data[i][3] = wire.getAttribute("");
    data[i][4] = wire.getAttribute("");
    data[i][5] = wire.getAttribute("");
    data[i][6] = wire.getAttribute("");

    int endNo = 0;
    for (IXWireEnd end : wire.getWireEnds())
    {
          if(end.getPin().getAttribute("Name") != null)
          {
                  String pinName = end.getPin().getAttribute("Name");
                  String pinListName =
                  end.getPin().getOwner().getAttribute("Name");


                  if (endNo == 0)
                  {
                  // check if wire ends for each wire is connected to a splice
                      if (end.getPin().getOwner() instanceof IXSplice)
                      {
                          IXSplice splice = (IXSplice) end.getPin().getOwner();
                         //check if the splice is Centre-strip
                         if (splice.isCenterstrip())
                         {
                            Set<IXWire> csWires = splice.getCenterstripWires();
                            for (IXWire csWire : csWires)
                             {
                               pinListName = csWire.getAttribute("Name");
                             }
                         }

                         else
                         pinListName = end.getPin().getOwner().getAttribute("Name");
                      }
                      else
                      pinListName = end.getPin().getOwner().getAttribute("Name");
                      data[i][3] = pinListName;
                      data[i][4] = pinName;

              endNo++;
                  }
                  else
                  {
                      if (end.getPin().getOwner() instanceof IXSplice)
                      {
                           IXSplice splice = (IXSplice) end.getPin().getOwner();

                         if (splice.isCenterstrip())
                         {

                                 Set<IXWire> csWires = splice.getCenterstripWires();
                                 for (IXWire csWire : csWires)
                                 {
                                 pinListName = csWire.getAttribute("Name");
                                 }
                         }
```

```
                                    else
                                    pinListName = end.getPin().getOwner().getAttribute("Name");
                                }
                                else
                                pinListName = end.getPin().getOwner().getAttribute("Name");
                                data[i][5] = pinListName;
                                data[i][6] = pinName;

                            }
                        }
                    }
                    data[i][7] = wire.getAttribute("OptionExpression");
                }
                i++;
            }
        }

        public int getColumnCount() {
            return 8; // The number of  columns
        }

        public int getRowCount() {
            return design.getConnectivity().getWires().size(); // The number of rows
        }


        public int getHeaderCount() {
            return 1;   // The number of headers
        }

        public Object getCellValueAt(int row, int column)
        {
           //Display data at each cell (Row * Column)
            Object obj = "x";
            switch (column) {
                case 0:
                case 1:
                case 2:
                case 3:
                case 4:
                case 5:
                case 6:
                case 7:
                    obj = data[row][column];
                    break;
            }
            return obj;
        }

        public Object getHeaderValueAt(int rowIndex, int colIndex)
        {
           //display  header value
            Object obj = null;
            switch (colIndex) {
                case 0:
                    obj = "Wire No";
                    break;
                case 1:
                    obj = "Col";
                    break;
                case 2:
```

```
                obj = "Size";
                break;
            case 3:
                obj = "From";
                break;
            case 4:
                obj = "Cav";
                break;
            case 5:
                obj = "To";
                break;
            case 6:
                obj = "Cav";
                break;
            case 7:
                obj = "Option";
                break;
        }
        return obj;
    }
}
```

You can copy and modify the example to implement your own plugin table if you wish.
The example provided may be different in future releases of Capital. If you do use this example, therefore, you need make a copy of this (we reserve the right to add, remove or modify this example).